

Web Security: Introduction

Mainack Mondal

CS60112
Spring 2025



Outline

- How the web works
 - The anatomy of a web page: HTTP
 - Cookies, HTTPS
 - Making web interactive
- A few tools
 - CURL
 - Burp suite
 - Python sockets

How the web works: HTTP

Two key components

- Your web browser send requests to a web server



Two key components

- Your web browser send requests to a web server



Client side

Language: HTML, javascript, CSS
Send requests, receive responses



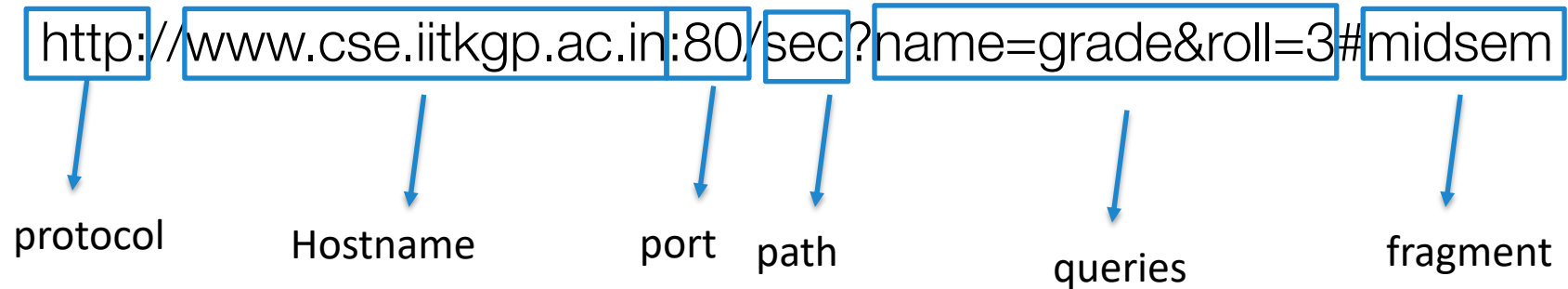
Developer side

Receives requests, send responses
Language: PHP, python (Django), Java...

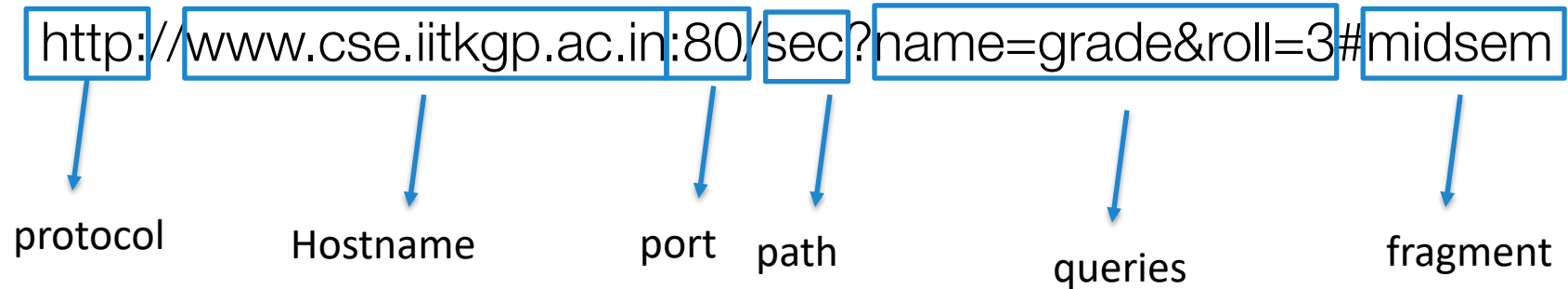
What protocol is used for requests?

- HTTP protocol
 - In the application layer
- HTTP
 - Simple
 - Stateless (meaning have no memory of earlier requests)
 - Unencrypted

User requests are in the form of URLs



User requests are in the form of URLs



- Special characters are encoded as hex in url:
%0A = newline
%20 or + = space, %2B = + (special exception)
Unicode ...

The setup: in a nutshell

- The developer sets up a server (e.g., for abcd.com)
 - Server runs a software like nginx or apache tomcat
 - Handles requests from web browser
- When a web browser makes a request to url *http://abcd.com/def*
 - First the web browser look up the ip of abcd.com
 - Sends a request to the ip with additional data (including the url)
 - Apache sees the url and then based on configuration files and codes take *action*
 - Request: GET and POST

The anatomy of a webpage

- view-source: <http://cse.iitkgp.ac.in/index.html>
- HTML (hypertext markup language)
 - Formatting of a page
 - All sorts of formatting: `<br />` `<div></div>` `<p></p>`
 - Links: `<a href="cse.iitkgp.ac.in">Click here</a>`
 - Pictures: ``
 - Forms
- HTML 5 introduced many media elements

Not all media elements are *safe*



The anatomy of a webpage

- CSS (cascading style sheets)
- `<link rel="stylesheet" type="text/css" href="./support/site.css">`
- [view-source:http://cse.iitkgp.ac.in/support/site.css](http://cse.iitkgp.ac.in/support/site.css)
- DOM (document object model)
 - object-based representation of the source HTML document
 - object structure of the DOM is represented by a “node tree”

```
<html>...  
  <body>  
    ...  
  </body>  
</html>
```

You type cse.iitkgp.ac.in in Chrome

- DNS (domain name service)
 - Resolves to IP address 203.110.245.250
- URL (uniform resource locator)
- https://cse.iitkgp.ac.in
 - Protocol: https
 - Hostname: cse.iitkgp.ac.in
 - Filename: index.html or similar (implicit)

HTTP Request

- Start line: method, target, protocol version
 - GET /index.html HTTP/1.1
 - Method: GET, PUT, POST, HEAD, OPTIONS
- HTTP Headers
 - Host, User-agent, Referer, many others
 - <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers>
 - Body (not needed for GET, etc.)

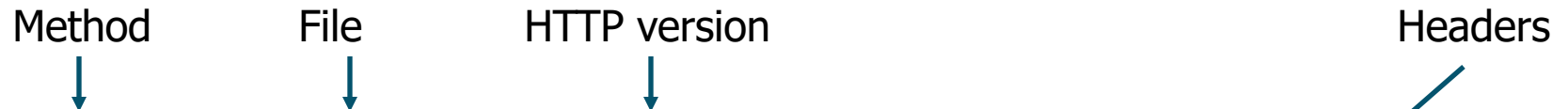
HTTP Request

Method

File

HTTP version

Headers



GET /index.html HTTP/1.1
Accept: image/gif, image/x-bitmap, image/jpeg, */*
Accept-Language: en
Connection: Keep-Alive
User-Agent: Mozilla/1.22 (compatible; MSIE 2.0; Windows 95)
Host: www.example.com
Referer: <http://www.google.com?q=dingbats>

The diagram shows an HTTP request structure. Labels with arrows point to specific parts: 'Method' points to 'GET', 'File' points to '/index.html', 'HTTP version' points to 'HTTP/1.1', 'Headers' points to the header section, 'Blank line' points to the empty line after the headers, and 'Data - none for GET' points to the empty space below the blank line.

Blank line

Data – none for GET

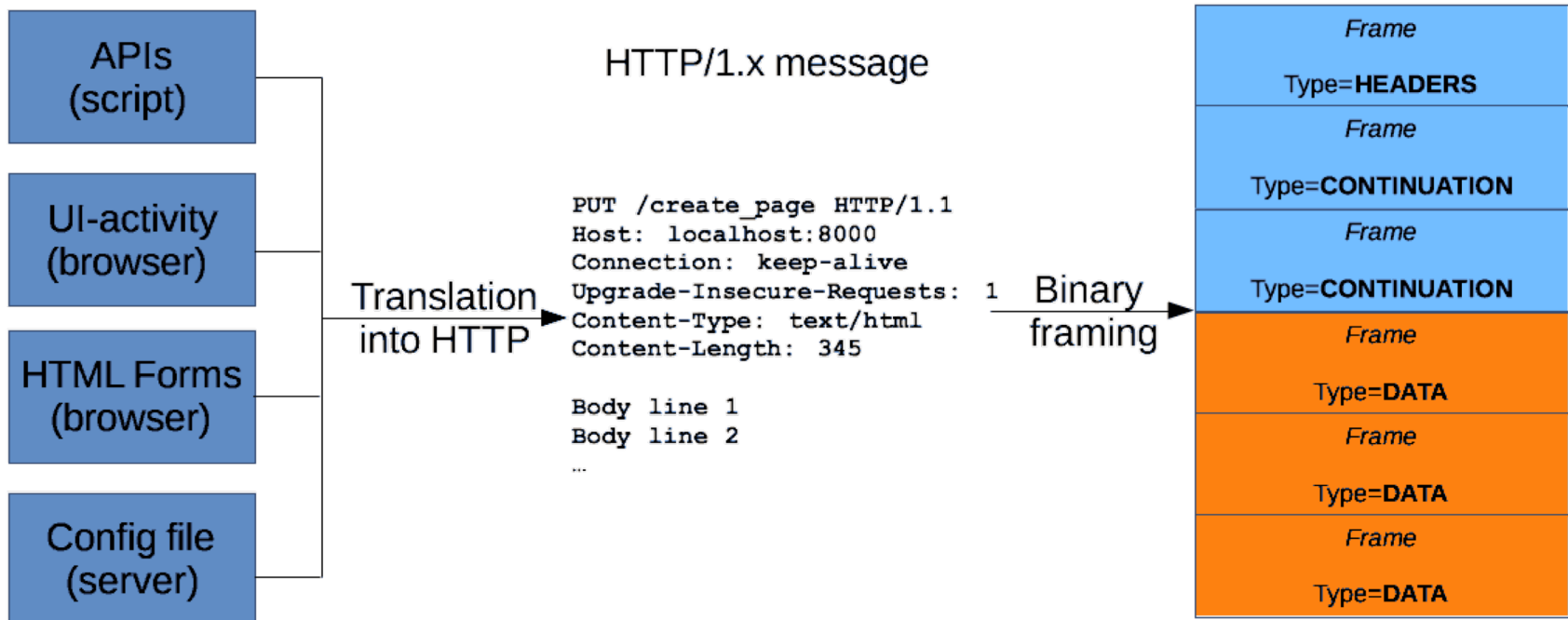
GET: no side effect.

POST: possible side effect.

HTTP Request

- GET /index.html HTTP/1.1

Activity initiation



From <https://developer.mozilla.org/en-US/docs/Web/HTTP/Messages>

HTTP Response

HTTP version

Status code

Reason phrase

Headers

```
HTTP/1.0 200 OK
Date: Sun, 21 Apr 1996 02:20:42 GMT
Server: Microsoft-Internet-Information-Server/5.0
Connection: keep-alive
Content-Type: text/html
Last-Modified: Thu, 18 Apr 1996 17:39:05 GMT
Set-Cookie: ...
Content-Length: 2543
```

Data

```
<HTML> Some data... blah, blah, blah </HTML>
```

Cookies

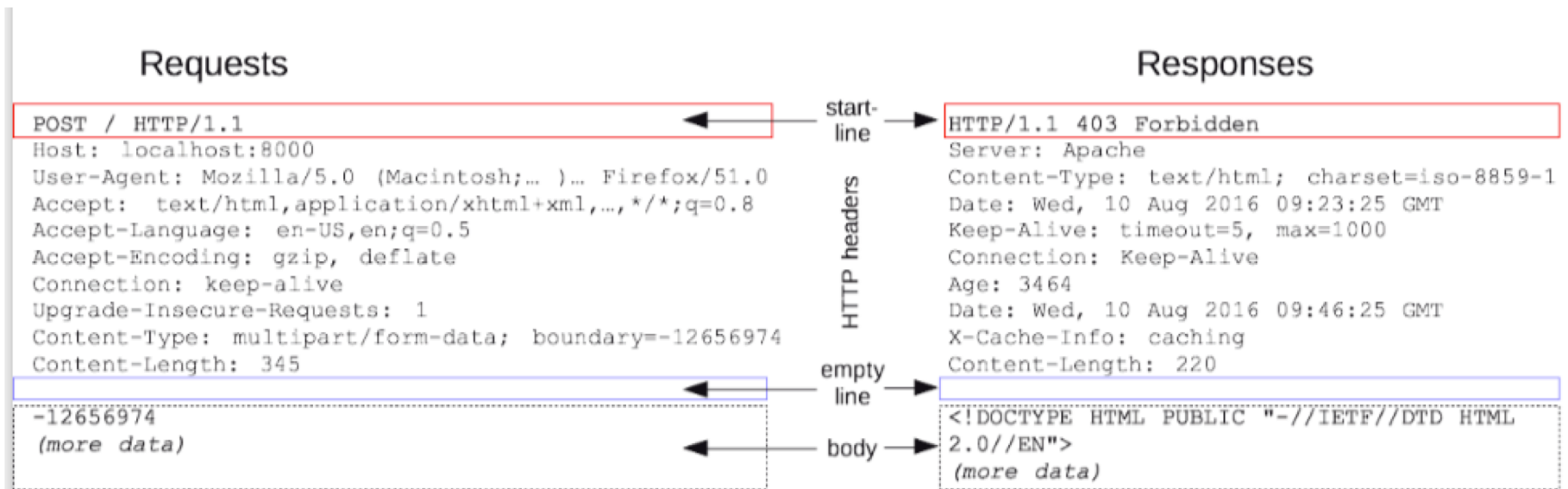
HTTP Response

- Status
 - 200 (OK)
 - 404 (not found)
 - 302 (redirect)
 - ...
- HTTP Headers
- Body

Let's have a demo

- Load up <http://www.iitkgp.ac.in/index.html> in browser
- Open up the developer console
 - In Chrome right click -> inspect element -> Network tab
 - Refresh the page
 - Click on the “index.html”

Putting it all together



Sending data to a server

- GET request
 - Data at end of URL (following “?”)
- POST request
 - Typically used with forms
 - Data not in URL, but rather (in slightly encoded form) in the HTTP request body
- PUT request
 - Store an entity at a location

Outline

- How the web works
 - The anatomy of a web page: HTTP
 - Cookies, HTTPS
 - Making web interactive
- A few tools
 - CURL
 - Burp suite
 - Python sockets

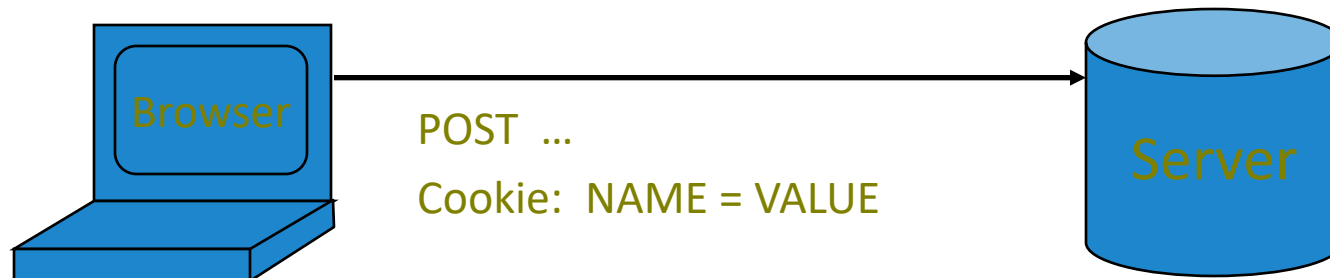
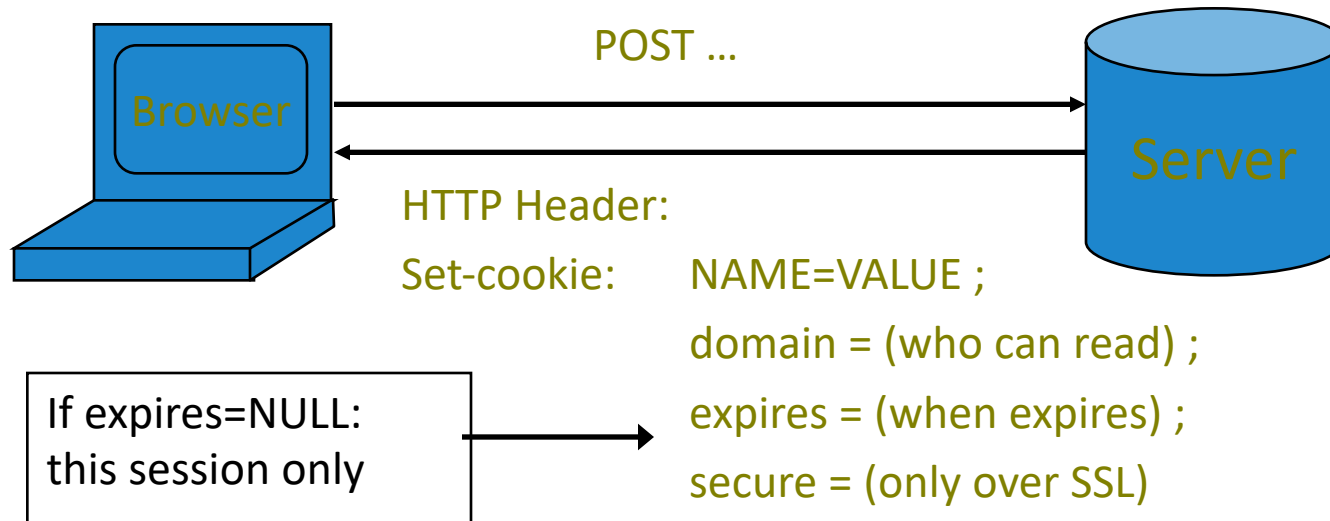
Improving HTTP: cookies and https

Recap: What protocol is used for requests?

- HTTP protocol
 - In the application layer
- HTTP
 - Simple
 - Stateless (meaning have no memory of earlier requests)
 - Unencrypted

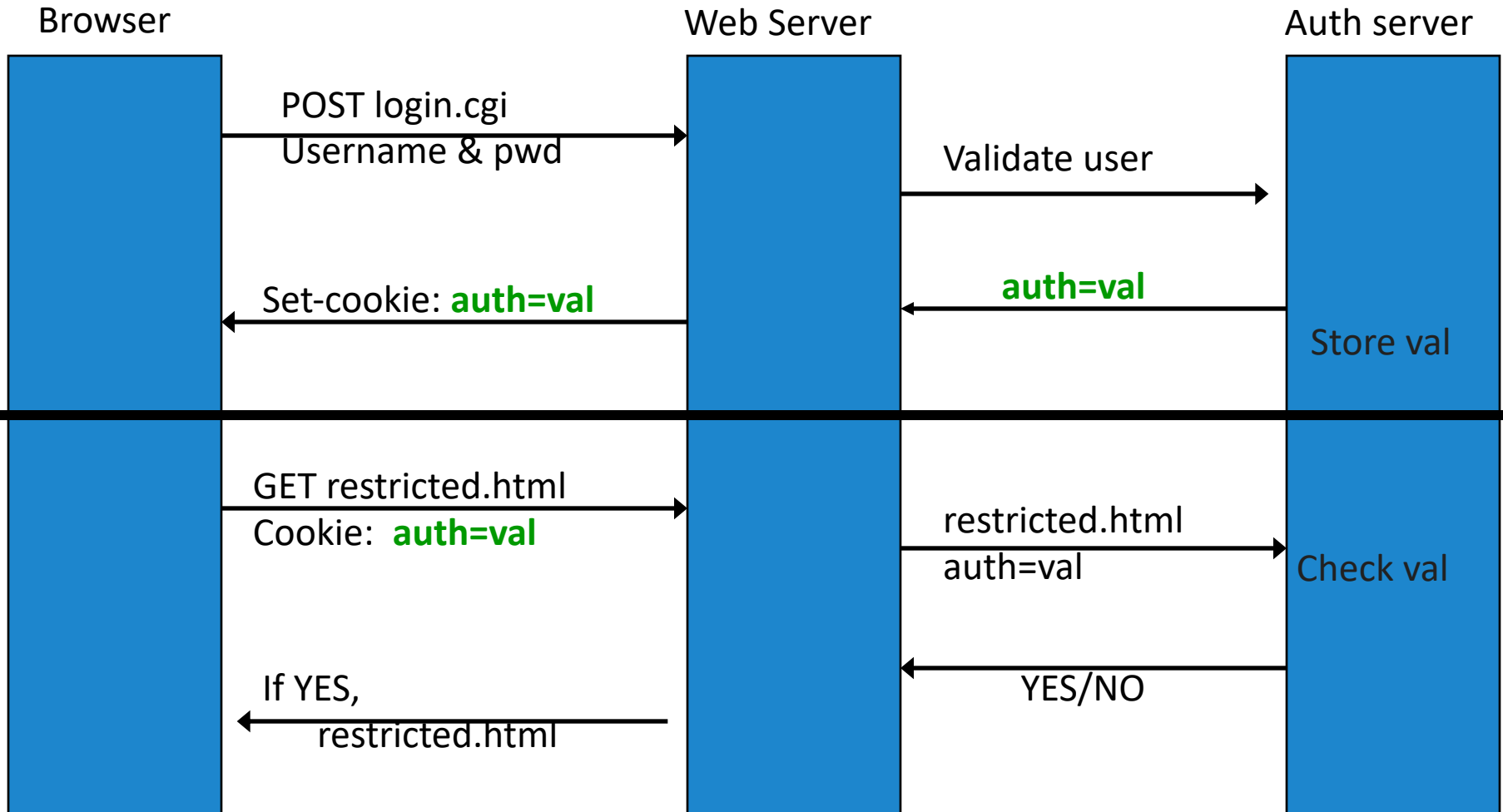
Cookies

- Used to store state on user's machine



HTTP is stateless protocol; cookies add state

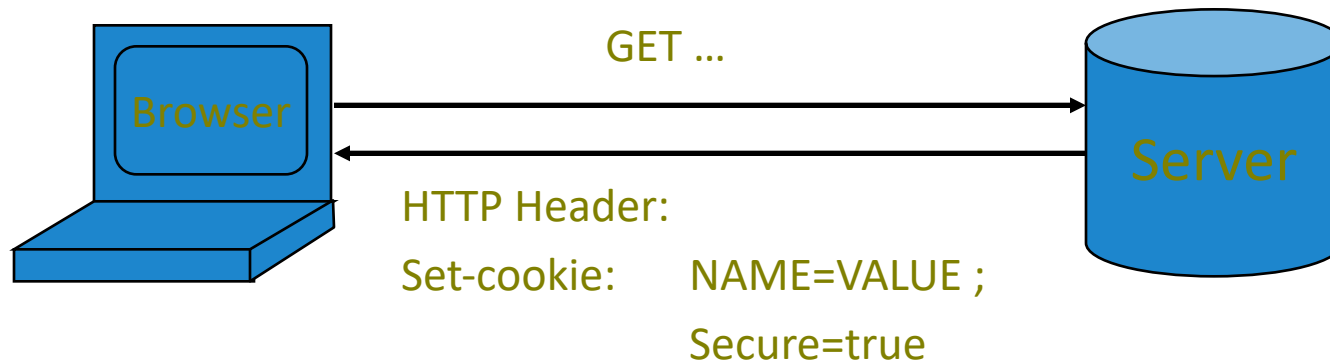
Cookie authentication



Cookie Security Policy

- Uses:
 - User authentication
 - Personalization
 - User tracking: e.g. Doubleclick (3rd party cookies)
- Browser will store:
 - At most 20 cookies/site, 3 KB / cookie (can be more)
- Origin is the tuple <domain, path>
 - Can set cookies valid across a domain suffix

Secure Cookies



- Provides confidentiality against network attacker
 - Browser will only send cookie back over HTTPS
- ... but no integrity
 - Can rewrite / set secure cookies over HTTP
 - ⇒ network attacker can rewrite secure cookies
 - ⇒ can log user into attacker's account

httpOnly Cookies



- Cookie sent over HTTP(s), but not accessible to scripts
 - cannot be read via `document.cookie`
 - Helps prevent cookie theft via XSS

... but does not stop most other risks of XSS bugs

Keeping state using cookies

Name	Value	Domain	Path	Expires / Max...	Size	H...	Secure	SameSite
Request Cookies					540			
1P_JAR	2019-09-12-10	N/A	N/A	N/A	19			
ANID	OPT_OUT	N/A	N/A	N/A	14			
APISID		N/A	N/A	N/A	43			
HSID	ASBk1141H7QW...	N/A	N/A	N/A	24			
NID		N/A	N/A	N/A	269			
SAPISID		N/A	N/A	N/A	44			
SEARCH_SAMESITE		N/A	N/A	N/A	26			
SID	hACB5Xgr7xjxLScuBTOhndborzhnGdWwA2zjslapyoaplast19...	N/A	N/A	N/A	77			
SSID		N/A	N/A	N/A	24			
Response Cookies					104			
1P_JAR	2019-09-12-10	.google.co.in	/	2019-10-12T...	104			none

Cookies: Basic summary

- HTTP is stateless
 - Amazon.in can not remember if you have already logged in
- Set-Cookie HTTP header
- Cookie HTTP header
 - Cookie: name=value; name2=value2; name3=value3
- Cookies, once stored locally, are automatically sent with all requests your browser makes
 - Check the cookie that <http://google.co.in> sets
- Session cookies vs. persistent cookies
 - Session: SSID (no expiration date)
 - Persistent: 1P_JAR (set by google, have expiration date)

HTTPS: real-world crypto

- HTTPS uses Transport Layer Security (TLS) for secure communication
 - Older version: Secure Sockets Layer (SSL)
 - TLS/SSL interchangeably used
 - Data transmitted via client-server “tunnel”
 - Much harder to compromise than HTTP
- A key component for secure communication with crypto
 - You need to share keys
 - Use certificates

Certificates

- To enable secured SSL connections, the server needs an SSL certificate signed by a Certificate Authority (CA).
 - CA verifies the ID of the certificate owner (e.g., cse.iitkgp.ac.in) when an SSL certificate is issued.
- Each SSL Certificate contains unique and authenticated information about the certificate owner
 - such as ID (in X.500 format), location, public key, and the signature of the CA
 - It confirms that you are who you say you are in the Internet
- An SSL certificate enables encryption of sensitive information during online transactions

HTTPS: Issues

- Which CAs (certificate authorities) does your browser trust?
- Firefox:
 - `about:preferences#privacy`
 - View certificates
- How do you know if a cert is still valid
 - Checking expiration dates
 - CRLs (certificate revocation lists), Cause unexpired != valid
 - Latest: OCSP (online certificate status protocol)

Outline

- How the web works
 - The anatomy of a web page: HTTP
 - Cookies, HTTPS
 - Making web interactive
- A few tools
 - CURL
 - Burp suite
 - Python sockets

**Making web interactive (and
insecure)**

Interactive pages – How?

- Javascript!
 - The core idea: Let's run (somewhat) arbitrary code on the client's computer
- Math, variables, control structures
- Imperative, object-oriented, or functional
- Can modify the DOM
- Request data (e.g., through AJAX)
- Can be multi-threaded (web workers)

Common javascript libraries

- JQuery (easier access to DOM)
 - `$(".test").hide()` hides all elements with class="test"
- JQueryUI
- Bootstrap
- Angular / React
- Google Analytics

But, javascript sometimes go to the dark side

- Client side read/write
 - `document.cookie`
 - Setting cookie: `document.cookie = "name=value; expires=...;`
 - Reading cookie: `alert(document.cookie)`
- Deleting a cookie is also possible
- Recall that a bank verifies that you logged in using cookies
 - What if a script running on another tab reads the cookie and sends it to an attacker?

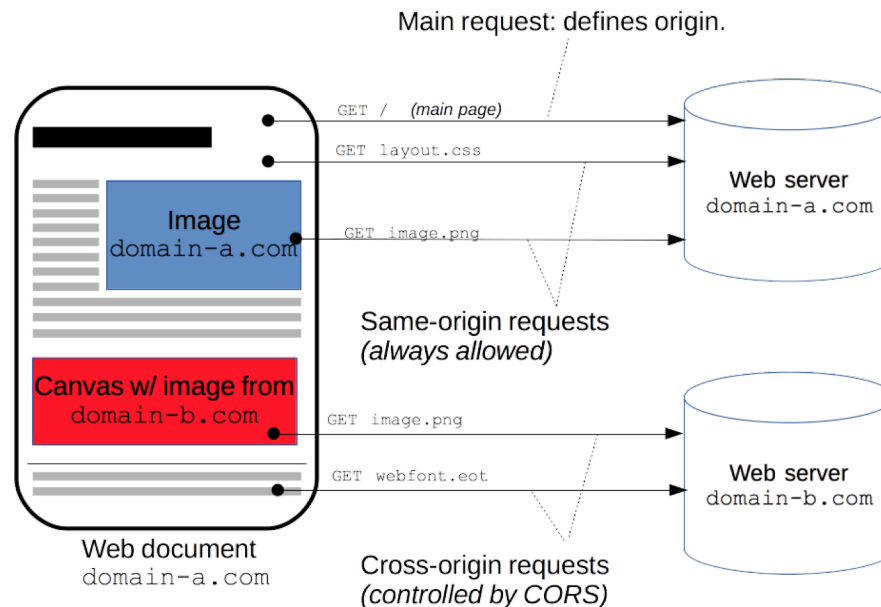
Same origin policy to the rescue

- Prevent malicious DOM access
- First, Origin = URI scheme, host name, port
- Only if origin that loaded a script matches then script can access the DOM
 - Not where the script comes from, but what origin loads the script
- Frames / iframes impact origin
 - bank.com can load script from attacker.com in an iframe for ads
- CORS (Cross-Origin Resource Sharing)
 - JavaScript code served from `https://a.com` can use XMLHttpRequest to make a request for `https://domain-b.com/data.json`

<https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>

Cross-Origin Resource Sharing (CORS)

- A server can indicate any origins (domain, scheme, or port) other than its own from which a browser should permit loading resources
- JS from `https://domain-a.com` uses *XMLHttpRequest* to make a request for `https://domain-b.com/data.json`

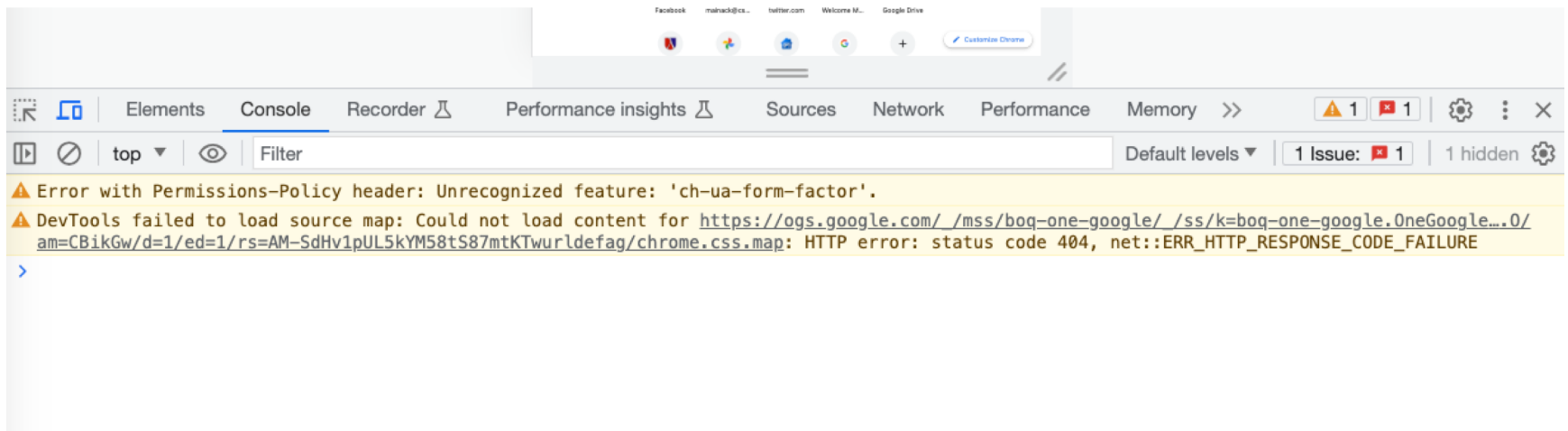


<https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>

Outline

- How the web works
 - The anatomy of a web page: HTTP
 - Cookies, HTTPS
 - Making web interactive
- A few tools
 - Curl
 - Burpstuite
 - Pyhton sockets

Inspect & browser console



Curl

- Command line tool
- Make http or https requests from command line
- Can have a zillion parameters
- Can emulate a request from a browser
- Cheat sheet: <https://devhints.io/curl>
- You can even copy request as curl from browser

burpsuite

- Browser based interface
- Check requests in a webpage
- Modify requests in a webpage
- Much more ...
- Required:
<https://portswigger.net/burp/documentation/desktop/getting-started>

Python sockets

- Python library for socket programming
- See the raw data coming and going from your machine
- Easy tutorial (not fully needed now) :
<https://www.pythonsheets.com/notes/python-socket.html>
- You can send specially crafted packets to a web server

Bonus: nc

- You want to send a non-http request
- Too lazy to do socket programming
- Use netcat or nc
- Cheatsheet: <https://quickref.me/nc.html>
- Knowing wireshark will be good...

Bonus: nmap

- You want to “know” about a server
- What ports are open? What OS? ...
- Use nmap
- Cheat sheet: <https://hackertarget.com/nmap-cheatsheet-a-quick-reference-guide/>